

Lec 1-4 - ชั่วโมง 30 ชั่วโมง } 40 ชั่วโมง  
 2 ชม. - อีก 3 ชั่วโมง  
 ก็รู้ว่าจะสอบทุกแบบแล้ว  
 ต้องรู้ไว้ด้วย

# Machine Learning

## Lecture 2: Training Models

9 ข้อควรระวัง  
 ข้อควรระวัง : Linear Regression, Bayesian,  
 MSE, Lagrange, ฯลฯ

- Calculator is allowed ✓
- Cheat Sheet is not allowed X

**Asst. Prof. Dr. Santitham Prom-on**

Department of Computer Engineering, Faculty of Engineering  
 King Mongkut's University of Technology Thonburi

# Topics

- Direct approach: linear regression with ordinary least square
- Iterative approach
  - Gradient descent
  - Batch gradient descent
  - Stochastic gradient descent
  - Mini-batch gradient descent
- Logistic regression

Notebook:

<https://colab.research.google.com/drive/1jcPGje7ymfGZLL9C2OV0VdqJQXWpe7h9?usp=sharing>

# Model training

## Two different ways

Aka Normal equation

► mathematical equation that gives the result directly

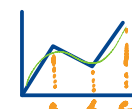
- Using a direct “closed-form” equation.

matrix

$$\begin{aligned} ax_1 + bx_2 &= y_1 \\ cx_1 + dx_2 &= y_2 \end{aligned}$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

วิธีนี้ใช้สมการปิด



คือ linear Regression

แบบปิดคือ close-form

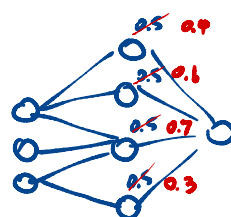
คือ polynomial degree 3 ก็ได้

ทำซ้ำ

- Using an iterative optimization approach.

called Gradient Descent (GD), that gradually tweaks the model parameters to minimize the cost function over the training set, eventually converging to the same set of parameters as the first method.

Ex: Neural Network



ทำซ้ำเพื่อหาค่าที่เหมาะสมที่สุด

# Direct approach

## Linear regression with Ordinary Least Square (OLS)

- Linear regression model can be expressed by the following formula

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

สมการที่เราได้มาคือ closed-form  
ด้วยนั้น ไม่กังวล  $\epsilon$

where

Real value  $\textcircled{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n + \epsilon$

$\hat{y}$  is the predicted value

ซึ่ง parameter ใหม: ค. ขอบเขตก็จ: เพิ่ม

$n$  is the number of features

$x_i$  is the  $i^{\text{th}}$  feature value

$\theta_j$  is the  $j$ th model parameter (including the bias term  $\theta_0$  and the feature weights  $\theta_1, \theta_2, \dots, \theta_n$ ).



# Simple linear regression (only one feature x)

$$Y_i = \alpha + \beta X_i + \varepsilon_i$$

$$e_i \sim N(0, \underbrace{\sigma^2}_{\text{Variance}}) \quad i.i.d.$$

$\varepsilon_i$  is independent of  $X_i$

- The intercept is  $\alpha$
- The slope is  $\beta$
- We use the normal distribution to describe the “error”

# Method of least squares

- Choose the  $\beta$ 's so that the sum of the squares of the errors,  $\varepsilon_i$ , are minimized
- The least squares function is

נוח  $\hat{y} = \alpha + \beta x$

נוח  $\varepsilon = y - \hat{y}$

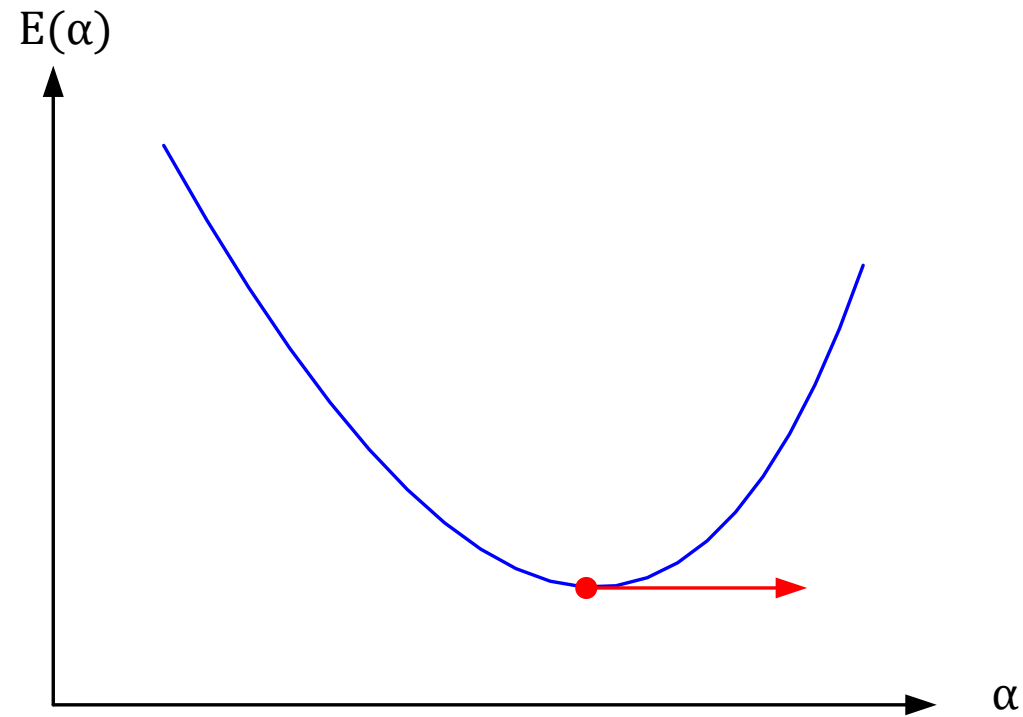
$$\varepsilon_i = y_i - \alpha - \beta x_i$$

$$\begin{aligned} \text{Sum } S &= \sum_{i=1}^n \varepsilon_i^2 \\ &= \sum_{i=1}^n \left( \overset{\text{Actual Val}}{y_i} - \underbrace{\alpha - \beta x_i}_{\hat{y}_i} \right)^2 \end{aligned}$$

$\sum_{i=1}^n (y_i - \hat{y}_i)^2$

# OLS solution

Minimum of a function is the point where the slope is zero



Note:  $S = \sum_{i=1}^n (y_i - \alpha - \beta x_i)^2$

$\frac{du^n}{dx} = nu^{n-1} \cdot \frac{du}{dx}$

# Derivative of the error functions

The function  $S$  is to be minimized with respect to  $\beta_0, \beta_1$

and

$$\frac{\partial S}{\partial \alpha} = -2 \sum_{i=1}^n (y_i - \alpha - \beta x_i) = 0$$

$$\begin{aligned} \frac{ds}{d\alpha} &= \frac{d}{d\alpha} \sum_{i=1}^n (y_i - \alpha - \beta x_i)^2 \\ &= \sum_{i=1}^n \frac{d}{d\alpha} (y_i - \alpha - \beta x_i)^2 \\ &= 2 \sum_{i=1}^n (y_i - \alpha - \beta x_i) \cdot \frac{d(y_i - \alpha - \beta x_i)}{d\alpha} \\ &= 2 \sum_{i=1}^n (y_i - \alpha - \beta x_i) \cdot \frac{d(y_i - \alpha - \beta x_i)}{d\alpha} \\ &= 2 \sum_{i=1}^n (y_i - \alpha - \beta x_i) (-1) \\ \text{Function } S \text{ to be minimized} \downarrow \\ 0 &= -2 \sum_{i=1}^n (y_i - \alpha - \beta x_i) \end{aligned}$$

$$\frac{\partial S}{\partial \beta} = -2 \sum_{i=1}^n (y_i - \alpha - \beta x_i) x_i = 0$$

$$\begin{aligned} \frac{ds}{d\beta} &= \frac{d}{d\beta} \sum_{i=1}^n (y_i - \alpha - \beta x_i)^2 \\ &= \sum_{i=1}^n \frac{d}{d\beta} (y_i - \alpha - \beta x_i)^2 \\ &= 2 \sum_{i=1}^n (y_i - \alpha - \beta x_i) \cdot \frac{d(y_i - \alpha - \beta x_i)}{d\beta} \\ &= 2 \sum_{i=1}^n (y_i - \alpha - \beta x_i) \cdot (-x_i) \\ \text{Function } S \text{ to be minimized} \downarrow \\ 0 &= -2 \sum_{i=1}^n (y_i - \alpha - \beta x_i) x_i \end{aligned}$$

# Least square normal equation

$$\left\{ \begin{array}{l} n\alpha + \beta \sum_{i=1}^n x_i = \sum_{i=1}^n y_i \\ \alpha \sum_{i=1}^n x_i + \beta \sum_{i=1}^n x_i^2 = \sum_{i=1}^n x_i y_i \end{array} \right.$$

$$0 = -2 \sum_{i=1}^n (y_i - \alpha - \beta x_i)$$

$$0 = \sum_{i=1}^n y_i - \sum_{i=1}^n \alpha - \sum_{i=1}^n \beta x_i$$

$$n\alpha + \beta \sum_{i=1}^n x_i = \sum_{i=1}^n y_i$$

$$0 = -2 \sum_{i=1}^n (y_i - \alpha - \beta x_i) x_i$$

$$0 = \sum_{i=1}^n y_i x_i - \alpha \sum_{i=1}^n x_i - \beta \sum_{i=1}^n x_i^2$$

$$\alpha \sum_{i=1}^n x_i + \beta \sum_{i=1}^n x_i^2 = \sum_{i=1}^n x_i y_i$$

# Find alpha (intercept) *Cramer's Rule*

The normal equation in matrix form are: 
$$\begin{bmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \end{bmatrix}$$

## Cramer's Rule

$$\begin{aligned} a_1x + b_1y + c_1z &= d_1 \\ a_2x + b_2y + c_2z &= d_2 \\ a_3x + b_3y + c_3z &= d_3 \end{aligned}$$

$$\text{Let } D = \begin{vmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{vmatrix}$$

If  $D \neq 0$  then

$$x = \frac{\begin{vmatrix} d_1 & b_1 & c_1 \\ d_2 & b_2 & c_2 \\ d_3 & b_3 & c_3 \end{vmatrix}}{D}$$

$$y = \frac{\begin{vmatrix} a_1 & d_1 & c_1 \\ a_2 & d_2 & c_2 \\ a_3 & d_3 & c_3 \end{vmatrix}}{D}$$

$$z = \frac{\begin{vmatrix} a_1 & b_1 & d_1 \\ a_2 & b_2 & d_2 \\ a_3 & b_3 & d_3 \end{vmatrix}}{D}$$

The normal equation 
$$\begin{bmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \end{bmatrix}$$

# Find alpha (intercept)

$$\alpha = \frac{\begin{vmatrix} \sum_{i=1}^n y_i & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i y_i & \sum_{i=1}^n x_i^2 \end{vmatrix}}{\begin{vmatrix} n & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{vmatrix}}} = \frac{\sum_{i=1}^n x_i^2 \sum_{i=1}^n y_i - \sum_{i=1}^n x_i y_i \sum_{i=1}^n x_i}{n \sum_{i=1}^n x_i^2 - \left( \sum_{i=1}^n x_i \right)^2}$$

# Find beta (slope)

The normal equation  $\begin{bmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \end{bmatrix}$

$$\beta = \frac{\begin{vmatrix} n & \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i y_i \end{vmatrix}}{\begin{vmatrix} n & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{vmatrix}} = \frac{n \sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \sum_{i=1}^n y_i}{n \sum_{i=1}^n x_i^2 - \left( \sum_{i=1}^n x_i \right)^2}$$



# Example: Multiple regression (Multiple feature x)

| $x_1$ | $x_2$ | $y$ |
|-------|-------|-----|
| 1     | 2     | 12  |
| 2     | 1     | 9   |
| 3     | 2     | 19  |
| 1     | 1     | 8   |

$$y = c_1x_1 + c_2x_2 + c_3$$

$$\begin{aligned} c_1 + 2c_2 + c_3 &= 12 \\ 2c_1 + c_2 + c_3 &= 9 \\ 3c_1 + 2c_2 + c_3 &= 19 \\ c_1 + c_2 + c_3 &= 8 \end{aligned}$$

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 1 & 1 \\ 3 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 12 \\ 9 \\ 19 \\ 8 \end{bmatrix}$$

# Pseudoinverse

**Y**  $N \times 1$  vector

**A**  $N \times M$  matrix, where  $M$  is the number of parameters

**B**  $M \times 1$  vector  $\begin{bmatrix} \gamma \\ \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \alpha & \alpha & \alpha \\ \alpha & \alpha & \alpha \\ \alpha & \alpha & \alpha \end{bmatrix} \begin{bmatrix} \alpha \\ \alpha \\ \alpha \end{bmatrix}$

$$\mathbf{Y} = \mathbf{AB}$$

$$\mathbf{AB} = \mathbf{Y}$$

$\times \mathbf{A}^T$ ;

$$\mathbf{A}^T \mathbf{AB} = \mathbf{A}^T \mathbf{Y}$$

$\times (\mathbf{A}^T \mathbf{A})^{-1}$ ;

$$(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{AB} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}$$

$$\boxed{\cancel{\mathbf{A}^T} (\cancel{\mathbf{A}^T})^{-1} \cancel{\mathbf{A}^T} \mathbf{AB}}$$

$$\mathbf{B} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}$$

# Python: pseudoinverse

```
A = np.array([[1,2,1],
               [2,1,1],
               [3,2,1],
               [1,1,1]])

print(A)
```

```
[[1 2 1]
 [2 1 1]
 [3 2 1]
 [1 1 1]]
```

```
B = np.array([12,9,19,8])
B.shape = (-1,1)
print(B)
```

```
[[12]
 [ 9]
 [19]
 [ 8]]
```

```
np.matmul(np.linalg.pinv(A),B)
```

```
array([[ 3. ],  c1
       [ 5.5],  c2
       [-1.5]]) c3
```

# Example: Multiple regression

| $x_1$ | $x_2$ | $y$ |
|-------|-------|-----|
| 1     | 2     | 12  |
| 2     | 1     | 9   |
| 3     | 2     | 19  |
| 1     | 1     | 8   |

 $\hat{y}$ 
 $(y - \hat{y})^2$ 
 $c_1 = 3$ 
 $c_2 = 5.5$ 
 $c_3 = -1.5$ 

12.5

0.25

10

1

18.5

0.25

6.5

2.25

$$y = c_1x_1 + c_2x_2 + c_3$$

$$MSE = \frac{1}{n} \sum E^2$$

$$= \frac{1}{4} (0.25 + 1 + 0.25 + 2.25)$$

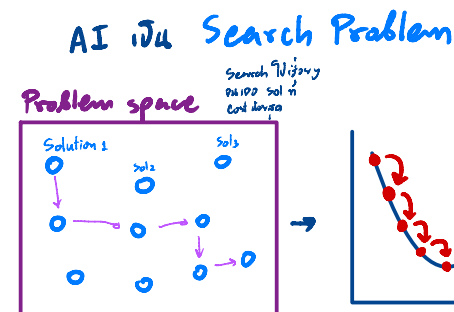
$$= \frac{3.75}{4} =$$

?

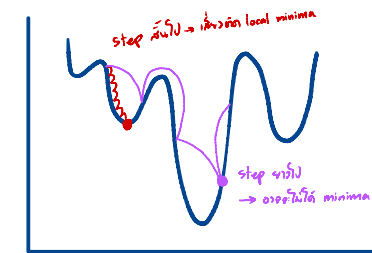
$$\begin{aligned} c_1 + 2c_2 + c_3 &= 12 \\ 2c_1 + c_2 + c_3 &= 9 \\ 3c_1 + 2c_2 + c_3 &= 19 \\ c_1 + c_2 + c_3 &= 8 \end{aligned}$$

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 1 & 1 \\ 3 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 12 \\ 9 \\ 19 \\ 8 \end{bmatrix}$$

# Iterative approach Gradient descent



การหาค่าต่ำสุด...



- A very generic optimization algorithm capable of finding optimal solutions to a wide range of problems.

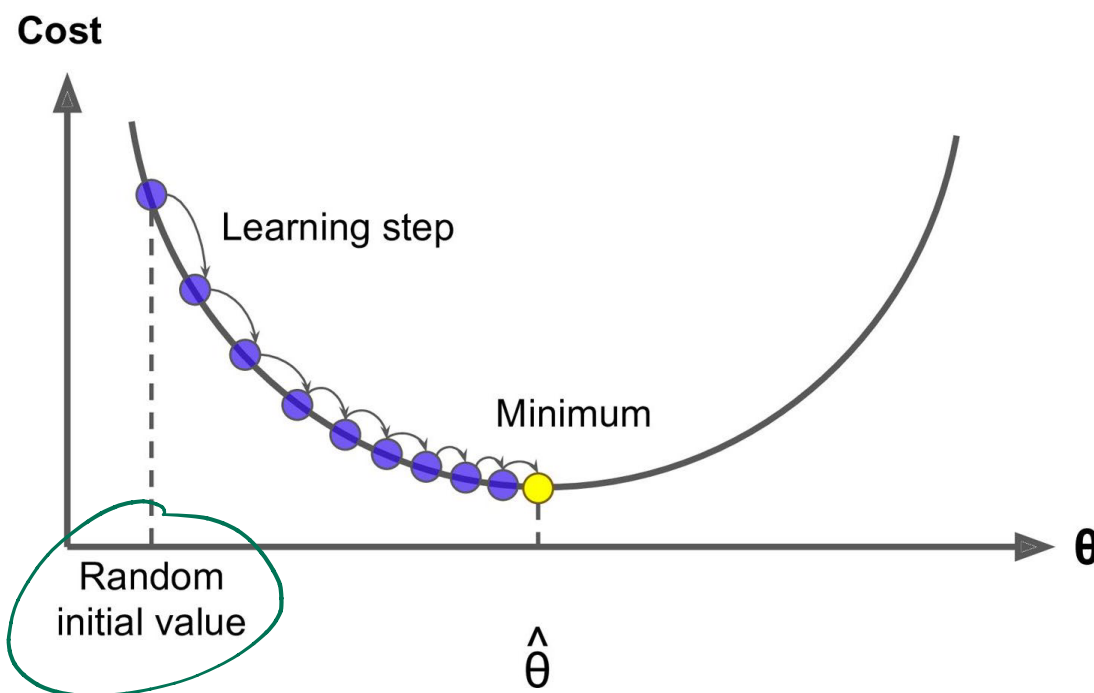


Figure 4-3. Gradient Descent

Suppose you are lost in the mountains in a dense fog; you can only feel the slope of the ground below your feet.

A good strategy to get to the bottom of the valley quickly is to go downhill in the direction of the steepest slope.

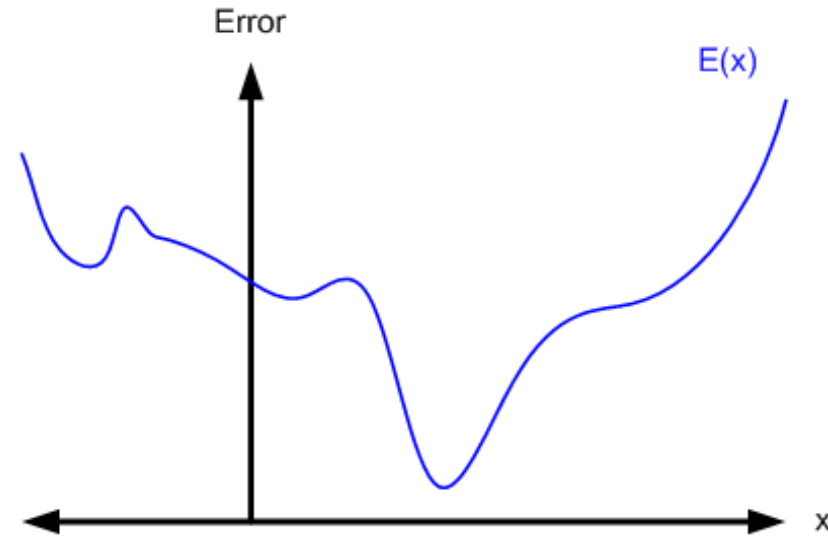
# Nonlinear Least Square

- Suppose that we have a sample of  $n$  observations on the response and the regressor, say,  $y_i, x_{i1}, x_{i2}, \dots, x_{ik}$  for  $i=1,2,\dots,n$
- The least square method involves minimizing the least square function

$$S(\boldsymbol{\beta}) = \sum_{i=1}^n \left[ \overset{\text{Real Val}}{y_i} - f(\overset{\text{Predicted Val}}{\underline{\mathbf{x}_i}}, \boldsymbol{\beta}) \right]^2$$

# Nonlinear objective function

Non-linear  
objective function



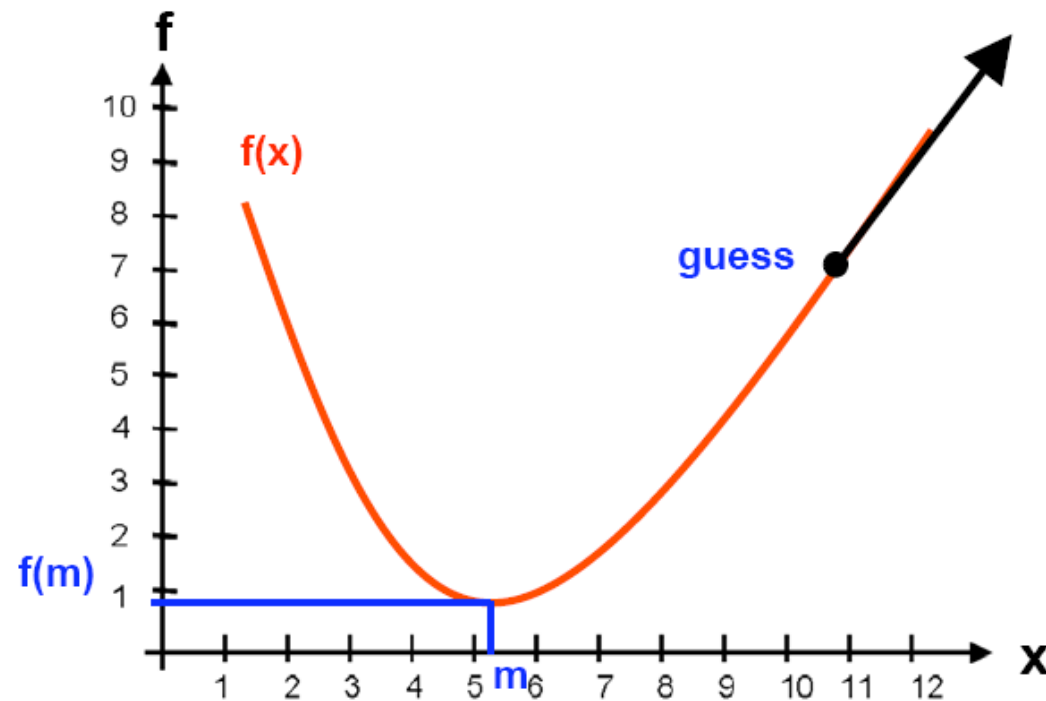
Unsolvable for roots  
of derivative of error

$$\frac{dE}{dx} = e^{-x}$$

# Gradient Descent Basic 1

## Objective function

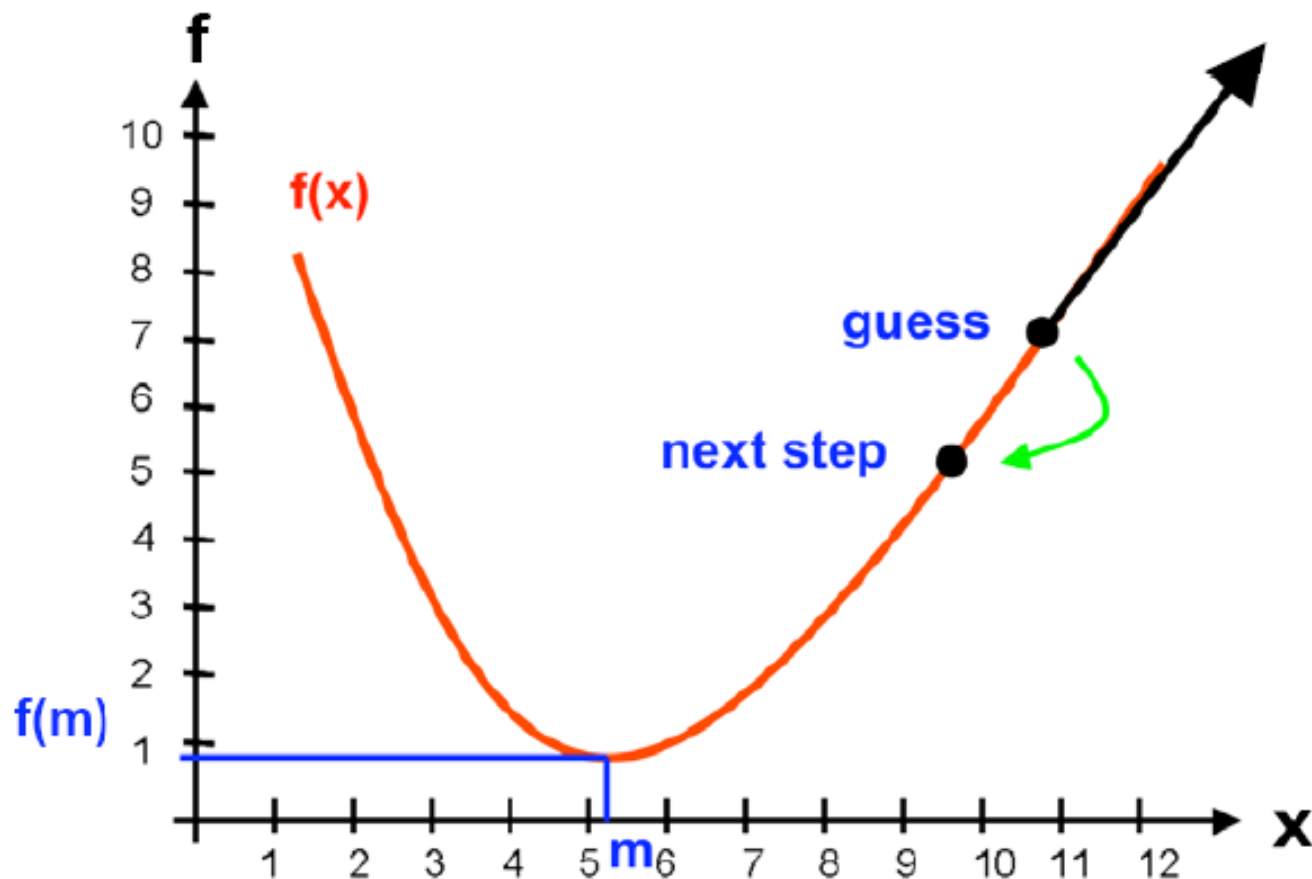
Minimum of a function is found by following the slope of the function





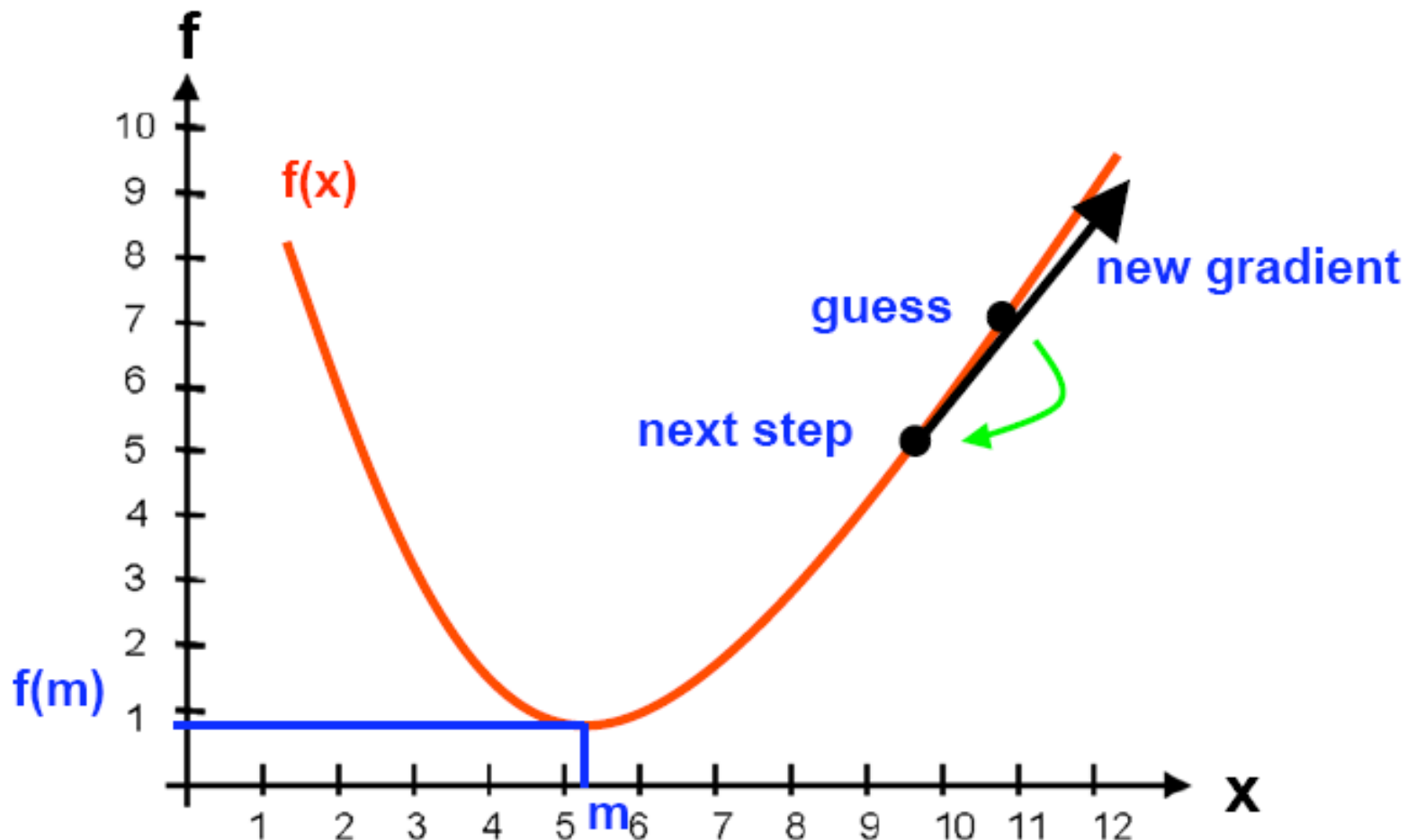
# Gradient Descent Basic 2

## Moving opposite to gradient



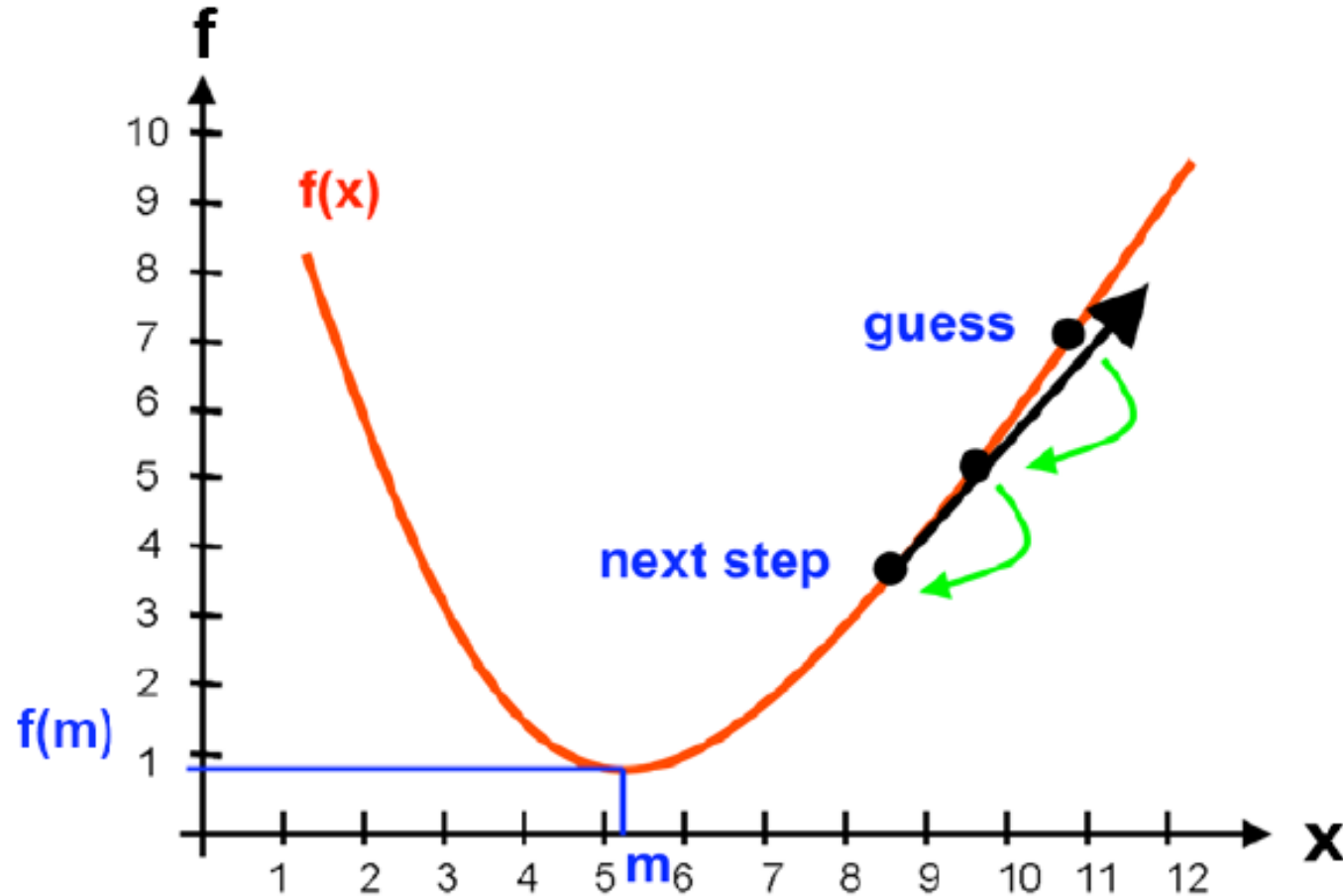
# Gradient Descent Basic 3

## Iterative gradient evaluation



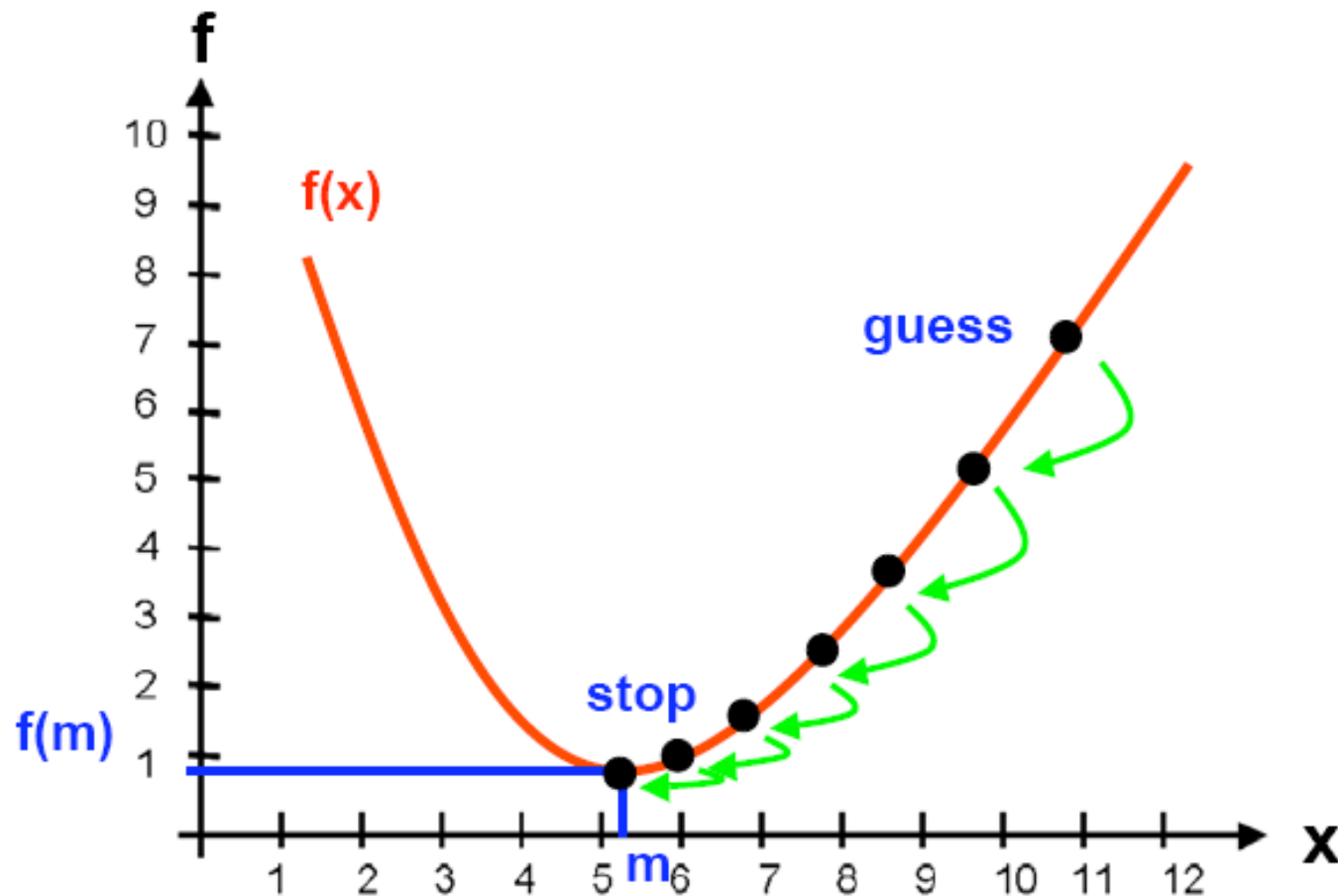
# Gradient Descent Basic 4

## Moving opposite to gradient



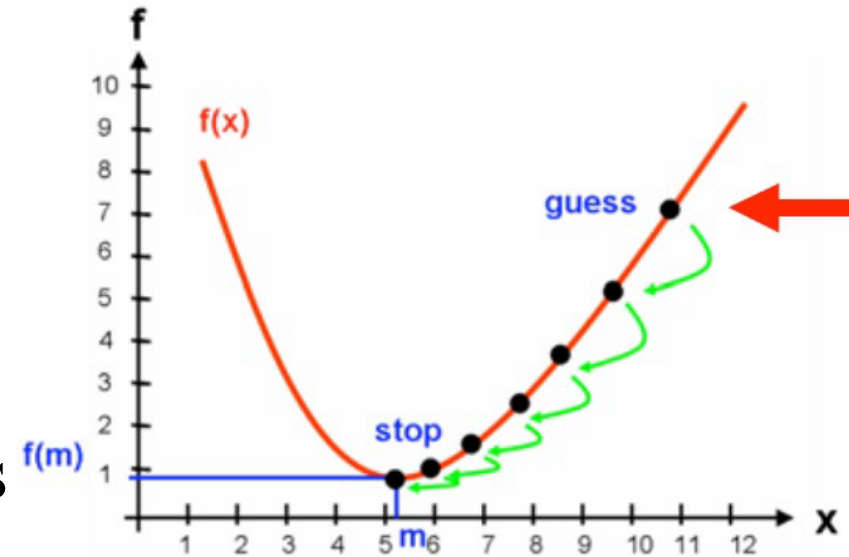
# Gradient Descent Basic 5

Iteratively descent opposite to the gradient



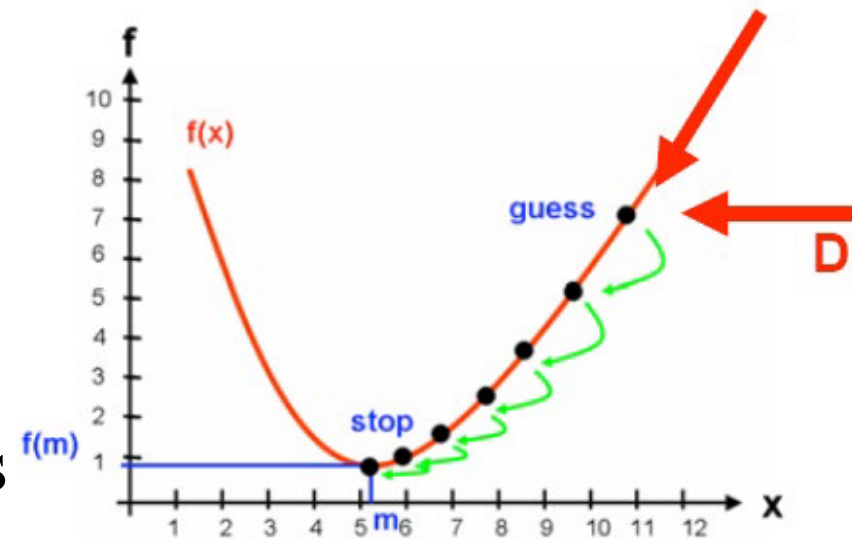
# Gradient Descent – algorithm

- Start with a point (randomly guessing)
- Repeat
  - Determine a descent direction
  - Choose a step
  - Update
- Until stopping criterion is satisfied



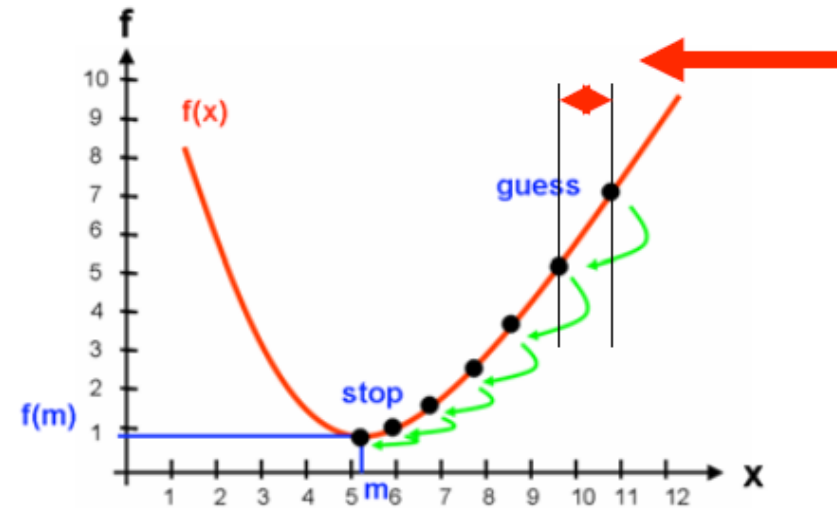
# Gradient Descent – algorithm

- Start with a point (randomly guessing)
- Repeat
  - Determine a descent direction
  - Choose a step
  - Update
- Until stopping criterion is satisfied



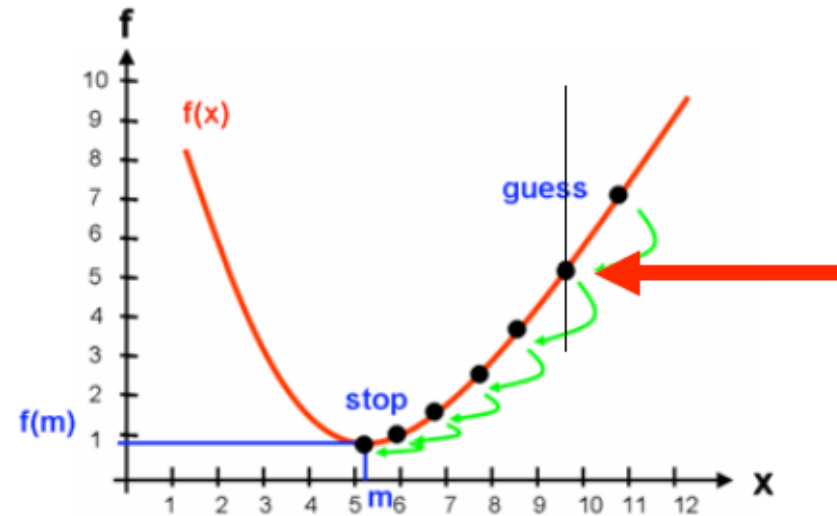
# Gradient Descent – algorithm

- Start with a point (randomly guessing)
- Repeat
  - Determine a descent direction
  - Choose a step
  - Update
- Until stopping criterion is satisfied



# Gradient Descent – algorithm

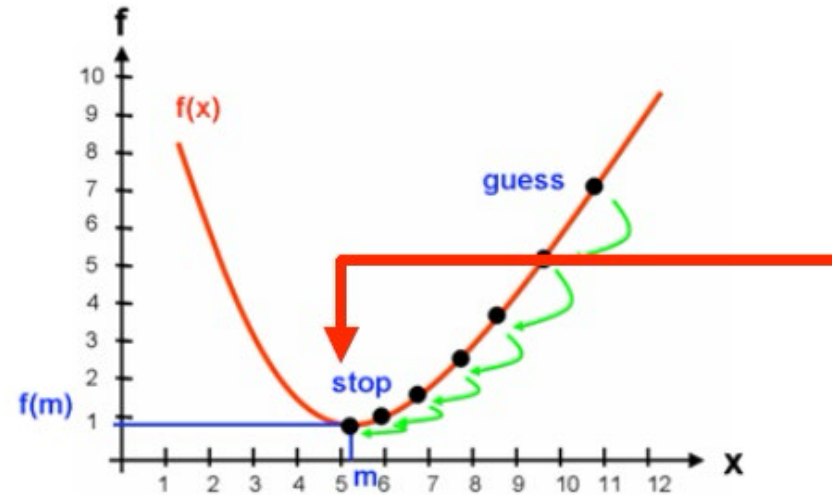
- Start with a point (randomly guessing)
- Repeat
  - Determine a descent direction
  - Choose a step
  - Update
- Until stopping criterion is satisfied





# Gradient Descent – algorithm

- Start with a point (randomly guessing)
- Repeat
  - Determine a descent direction
  - Choose a step
  - Update
- Until stopping criterion is satisfied



# Gradient Descent – algorithm

- Start with a point (randomly guessing) → Randomly guessing  $\beta$
- Repeat
  - Determine a descent direction →  $\text{direction} = -\frac{dS(\beta)}{d\beta}$
  - Choose a step →  $\text{step} > 0$
  - Update →  $\beta^{t+1} = \beta^t - \text{step} \frac{dS(\beta)}{d\beta}$
- Until stopping criterion is satisfied →  $\frac{dS(\beta)}{d\beta} \approx 0$

Compute the gradient using all training examples before updating param  
 ใช้รวม data point (x) ทั้งหมด

(คำนวณ & η on datapoint)  
 คำนวณรวมก่อน update

Pro - Uses exact gradient from the full dataset (stable convergence)  
 - ใช้รวม dataset ทีเดียว

Cons - ช้า  
 - ใช้ mem

# Batch Gradient Descent

- To implement Gradient Descent, you need to compute the gradient of the cost function with regards to each model parameter  $\theta_j$ .
- Batch gradient descent uses data of the whole batch to compute gradient

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\theta) = \frac{2}{m} \sum_{i=1}^m (\theta^T \mathbf{x}^{(i)} - y^{(i)}) \underline{x_j^{(i)}}$$

derivative  
w.r.t  $\theta_j$

$$\text{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$$

$m$  : # training samples  
 $\mathbf{x}^{(i)}$  : feature vector for sample  $i$   
 $y^{(i)}$  : actual output  
 $\theta$  : parameter vector (weight)

weight error  
by how much that feature affects it

$$\left[ \frac{\partial \text{MSE}}{\partial \theta_1}, \frac{\partial \text{MSE}}{\partial \theta_2}, \dots \right]$$

Update rule :

$$\theta^{(\text{next step})} = \theta - \eta \nabla_{\theta} \text{MSE}(\theta)$$

Vector of partial derivatives for all parameters.

Learning rate  
(controls step size)

# Stochastic gradient descent

- Batch Gradient Descent uses the whole training set to compute the gradients at every step, which makes it very slow when the training set is large.
- Stochastic gradient descent samples random instances for training at each training step

บทที่ ๑: ค่าความทุกจุด ให้สุ่มมาค่าความ 1 จุด แล้ว update เสร็จ

Pros เร็วสุด

Cons อาจจะไม่เวิร์ก

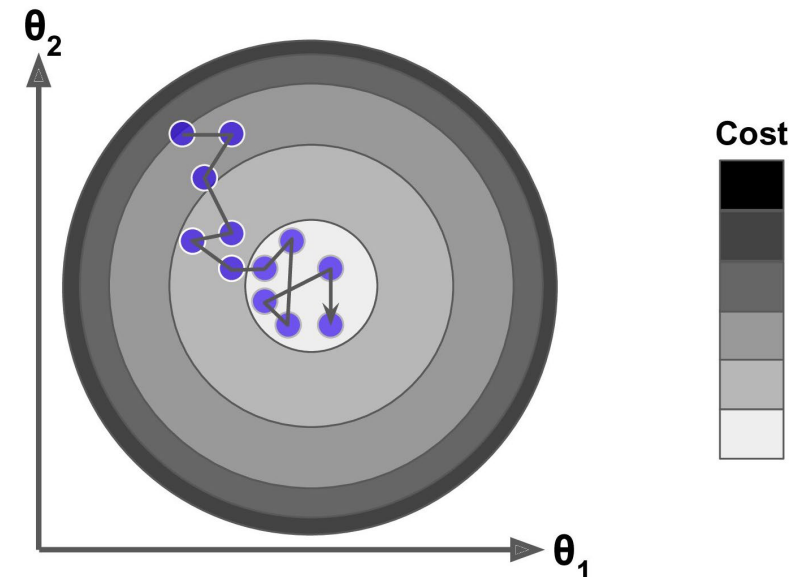


Figure 4-9. Stochastic Gradient Descent

# Mini-Batch Gradient Descent

Sample มาจำนวนหนึ่ง  
แล้ว update

- Both stochastic and use small batch
- Apply for small batch instead of an instance.
- Faster by GPU computing

---

**Algorithm 8.1** Stochastic gradient descent (SGD) update at training iteration  $k$

---

**Require:** Learning rate  $\epsilon_k$

**Require:** Initial parameter  $\theta$

**while** stopping criterion not met **do**

    Sample a minibatch of  $m$  examples from the training set  $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$  with corresponding targets  $\mathbf{y}^{(i)}$ .

    Compute gradient estimate:  $\hat{\mathbf{g}} \leftarrow +\frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$ .

    Apply update:  $\theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$ .

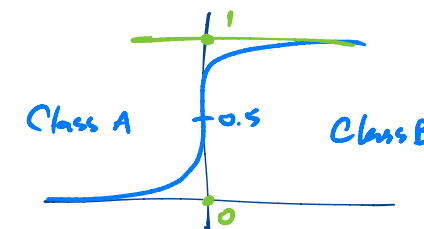
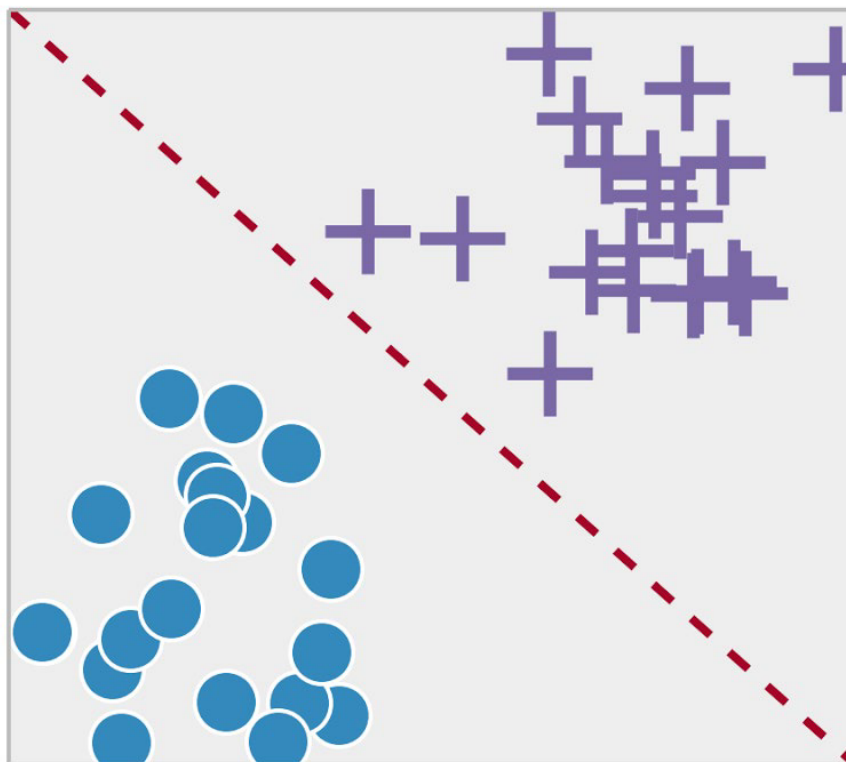
**end while**

---

based on Sigmoid function

# Example: Logistic Regression

Classification



Classification ၂

ကဲ probability ကဲ as ဒဲး class  
ကဲ probability ကဲ as ဒဲး class

# Logistic regression

Logistic regression has the following mathematical formulae,

$$\log \left( \frac{H_{\theta}(x_i)}{1 - H_{\theta}(x_i)} \right) = f(x_i) = \theta_0 + \sum_{i=1}^n \theta_i x_i$$

and

$$H_{\theta}(x_i) = \frac{1}{1 + e^{-f(x)}}.$$

# Loss function

This function has a nice property that,

$$\frac{\partial}{\partial \boldsymbol{\theta}} H_{\boldsymbol{\theta}}(x_i) = H_{\boldsymbol{\theta}}(x_i)(1 - H_{\boldsymbol{\theta}}(x_i))x_i.$$

For logistic regression, we can formulate the loss function as follows,

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n [-y_i \log H_{\boldsymbol{\theta}}(x_i) - (1 - y_i) \log (1 - H_{\boldsymbol{\theta}}(x_i))].$$





# Gradient

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta} &= \frac{\partial}{\partial \theta} \left( \frac{1}{n} \sum_{i=1}^n (-y_i \log H_{\theta}(x_i) - (1 - y_i) \log (1 - H_{\theta}(x_i))) \right) \\&= \frac{1}{n} \sum_{i=1}^n \left( -y_i \frac{\partial}{\partial \theta} \log H_{\theta}(x_i) - (1 - y_i) \frac{\partial}{\partial \theta} \log (1 - H_{\theta}(x_i)) \right) \\&= \frac{1}{n} \sum_{i=1}^n \left( -\frac{y_i}{H_{\theta}(x_i)} \frac{\partial}{\partial \theta} H_{\theta}(x_i) - \frac{1 - y_i}{1 - H_{\theta}(x_i)} \frac{\partial}{\partial \theta} (1 - H_{\theta}(x_i)) \right) \\&= \frac{1}{n} \sum_{i=1}^n \left( -\frac{y_i}{H_{\theta}(x_i)} H_{\theta}(x_i) (1 - H_{\theta}(x_i)) x_i \right. \\&\quad \left. - \frac{1 - y_i}{1 - H_{\theta}(x_i)} H_{\theta}(x_i) (1 - H_{\theta}(x_i)) (-x_i) \right) \\&= \frac{1}{n} \sum_{i=1}^n \left( -y_i (1 - H_{\theta}(x_i)) x_i - (1 - y_i) H_{\theta}(x_i) (-x_i) \right) \\&= \frac{1}{n} \sum_{i=1}^n \left( -y_i (1 - H_{\theta}(x_i)) + (1 - y_i) H_{\theta}(x_i) \right) x_i \\&= \frac{1}{n} \sum_{i=1}^n \left( -y_i + y_i H_{\theta}(x_i) + H_{\theta}(x_i) - y_i H_{\theta}(x_i) \right) x_i \\&= \frac{1}{n} \sum_{i=1}^n \left( \overset{\text{Predict val}}{H_{\theta}(x_i)} - \underset{\text{Real val}}{y_i} \right) x_i\end{aligned}$$

# Gradient descent

overfit adaptive learning rate with learning step decay also

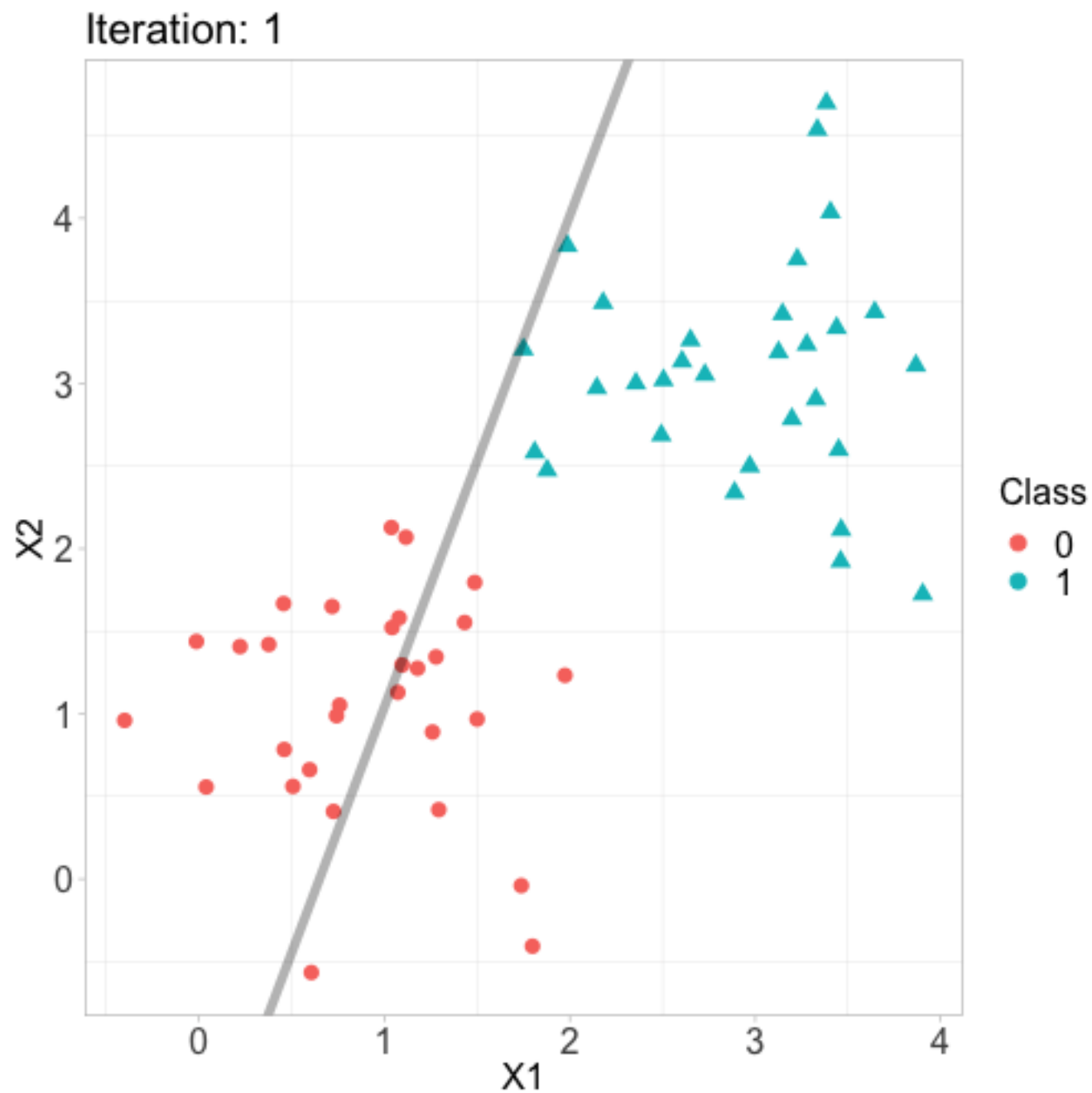
We can then iteratively update the parameters using the gradient descent approach,

Gradient desc - (Learning step) (Gradient  $\nabla_{\theta}$ )

$$\begin{aligned}\theta^{(k+1)} &= \theta^{(k)} - \alpha \frac{\partial J\theta}{\partial \theta} \\ &= \theta^{(k)} - \alpha \left( \sum_{i=1}^n (H_{\theta^{(k)}}(x_i) - y_i) x_i \right),\end{aligned}$$

where  $\alpha$  is the learning rate. The initial parameters  $\theta^{(0)}$  can be randomized or set to any values as the loss function is convex.

# Results





# End of Lecture 3

## Question?

